

Allocazione della Memoria

`static, global, extern, allocazione
statica/automatica/dinamica`

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio



Variabili Locali Static

Variabili Locali `static`



- Una variabile locale (cioè definita all'interno di una funzione)
 - Non mantiene il suo valore tra una chiamata e l'altra ... a meno di usare `static`
 - Se dichiarata `static` mantiene il suo valore anche dopo che la funzione è terminata

- Esempio:

```
int nextNumber() {  
    static int counter = 1;  
    return counter++;  
}
```

- `nextNumber()` restituisce:
 - la prima volta che è chiamata → 1
 - la seconda volta che è chiamata → 2
 - ...
 - l'n-esima volta che è chiamata (a meno di overflow) → n

Variabili Globali extern e static

Variabili Globali



Variabili globali
sono da usare
con parsimonia*

- Una variabile globale
 - È una variabile definita al livello più alto (top level),
 - Fuori da ogni blocco del file `.c`
 - Una variabile globale, e.g., `int v`, è accessibile:
 - All'interno dello stesso file: da dove è stata dichiarata in poi
 - Da altri file: se dichiarata come `extern int v`; in quel file
 - Nota: `extern int v` può precedere una dichiarazione `int v` nello stesso file
 - `extern int v`; significa che la variabile `int v` è definita in seguito: o nello stesso file oppure in un altro file.
 - Solo all'interno dello stesso file, se dichiarata `static int v`
 - Nota: `static` nasconde la variabile
- La visibilità delle funzioni è gestita nello stesso modo
 - Nota: per i prototipi di funzioni il qualificatore `extern` è implicito, quindi non serve scriverlo

*Non dovrebbero servire per i programmi che vedremo a ProglI

Memoria Assegnata ad un Programma C in Esecuzione

Tipi di Memoria Dati in C I



- La macchina astratta C usa tre tipi di memoria per allocare le variabili (i dati):
 1. **Memoria statica**
 - Usata per memorizzare le variabili globali e le variabili locali `static`
 2. **Memoria automatica (stack)**
 - Usata per memorizzare i record di attivazione delle funzioni (indirizzo di ritorno, parametri della funzione e le variabili locali (non `static`))
 3. **Memoria dinamica (heap)**
 - Usata per allocare variabili (spazi di memoria) la cui dimensione si conosce solo in fase di esecuzione

Tipi di Memoria Dati in C II

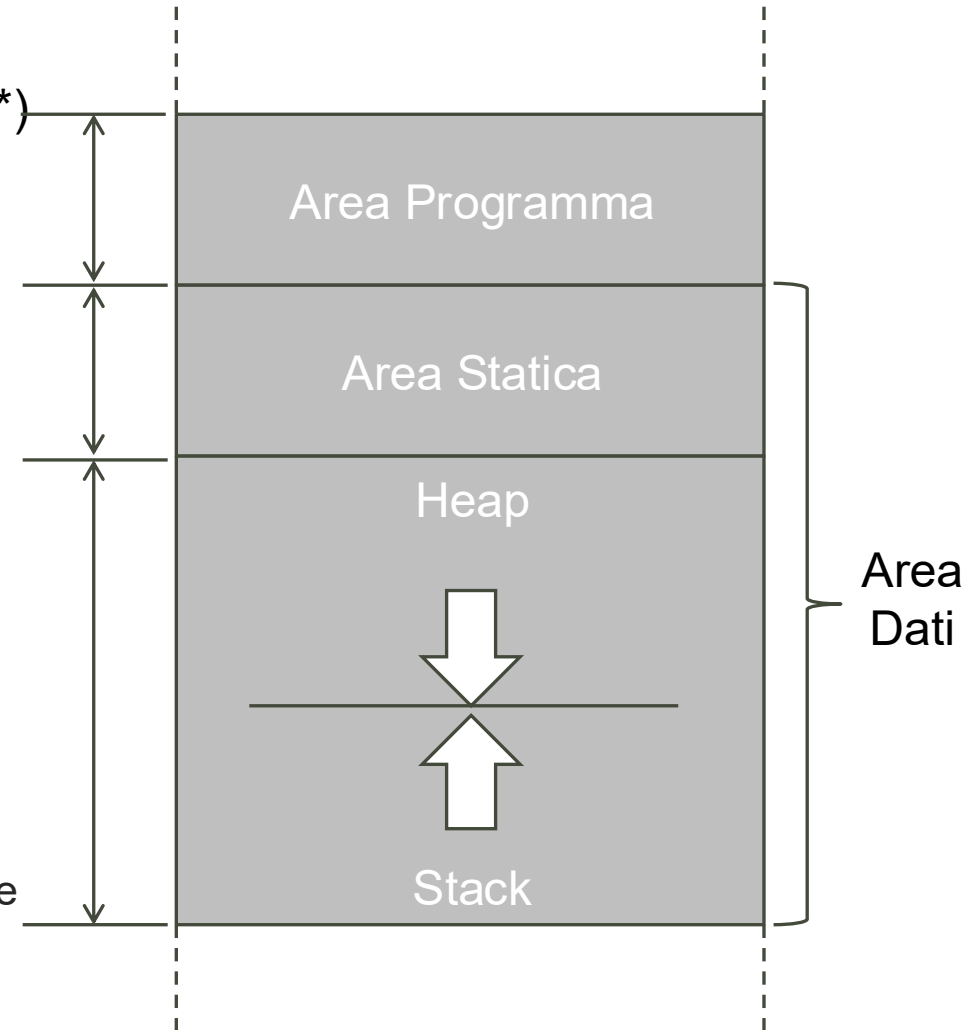


- Memoria statica e automatica (stack)
 - Gestiti in maniera implicita
- Memoria dinamica (heap)
 - Gestita in maniera esplicita
 - L'allocazione e la liberazione sono esplicite: cioè con apposite chiamate a funzioni
 - La funzione `malloc()` della libreria `stdlib.h` alloca uno spazio di memoria contiguo nell'heap e restituisce un puntatore al primo byte allocato.
 - In caso di fallimento, restituisce `NULL`, che corrisponde all'indirizzo riservato 0.
 - La funzione `free()` della libreria `stdlib.h` libera lo spazio di memoria precedentemente allocato con `malloc()`

La Memoria di un Programma C in Esecuzione



- Area Programma:
 - Memorizza il programma compilato e linkato (dimensione fissa*)
- Area Statica:
 - Memorizza le variabili globali **allocate staticamente** quando inizia l'esecuzione del programma (dimensione fissa*)
- Heap:
 - Memorizza le variabili **allocate dinamicamente** (dimensione varia durante l'esecuzione) (ma STACK + HEAP è fisso*)
- Stack:
 - Memorizza le variabili dichiarate all'interno delle funzioni attive
 - **Allocate automaticamente:** aggiungendo/togliendo un record di attivazione in cima allo stack, quando l'esecuzione di una funzione inizia/termina (dimensione varia durante l'esecuzione)



*Dimensione decisa a tempo di compilazione

Allocazione Dinamica

Allocazione e Deallocazione Dinamica



- Memoria dinamica (heap)

- Gestita esplicitamente durante l'esecuzione del programma
- Tramite funzioni della libreria `stdlib.h`

- Allocazione:

- `void *malloc(size_t size) ;`
- Alloca dinamicamente una quantità di memoria specificata da `size`
 - Restituisce `NULL` in caso di errore.

- Deallocazione:

- `void free(void *ptr) ;`
- Libera la memoria allocata in precedenza con `malloc()`: dopo `free`, la memoria è riallocabile.

```
cast-type *ptr;  
...  
ptr = malloc(byte-size);  
free(ptr);
```

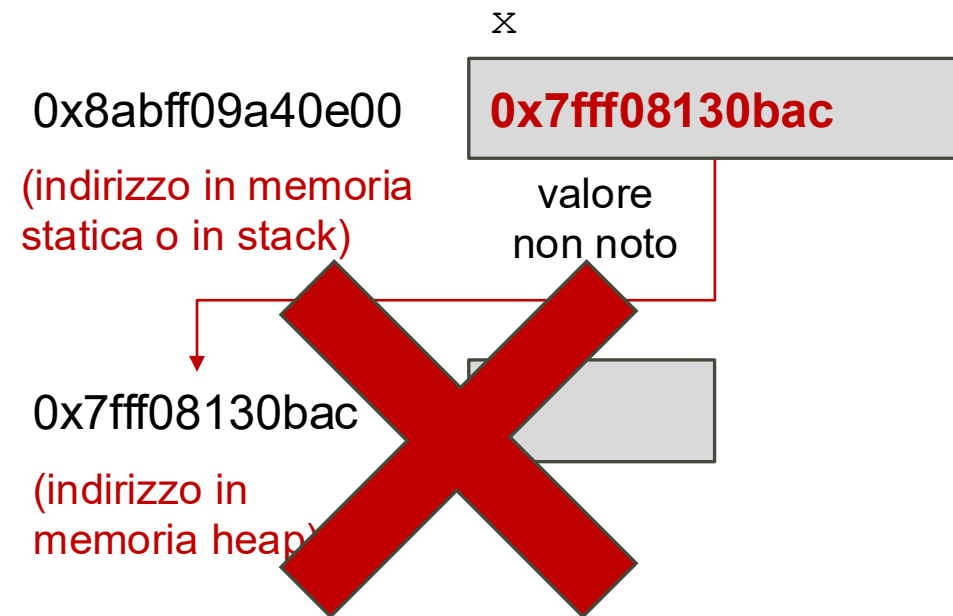
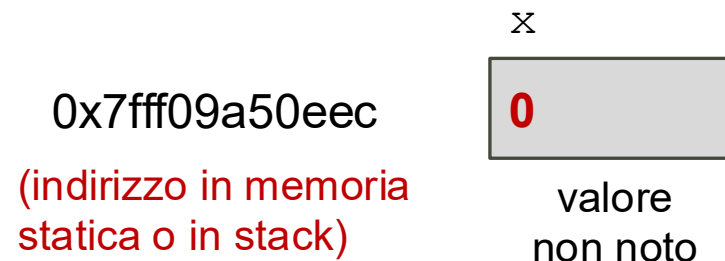
casting non
necessario e
ridondante

Allocazione Statica/Automatica Vs Dinamica

Allocazione di Variabile Semplice



Statica/Automatica	Dinamica
<pre>int x; // int x = 0;</pre>	<pre>int *x; //indirizzo di int x = malloc(sizeof(*x)); *x = 0; free(x);</pre>



Confronto Schematico



Statica/Automatica: meno tempo	Dinamica: meno spazio
→ Accesso più rapido → Occupo memoria anche per variabili che non uso	→ Accesso meno rapido → Occupo solo la memoria che mi serve

- **Esempio:**

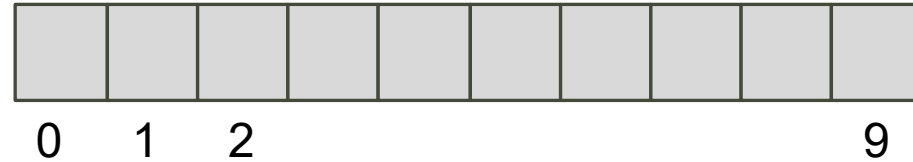
```
int array[100];
```

```
// dimensione fissa ma magari uso solo le prime 5 celle
```

Allocazione Array

Allocazione Statica/Automatica Array

```
int v[10];
```



- Quanto spazio occupa?

```
sizeof(v) == sizeof(int)*10 == 40 byte
```

- Posso scrivere `v+1` ?

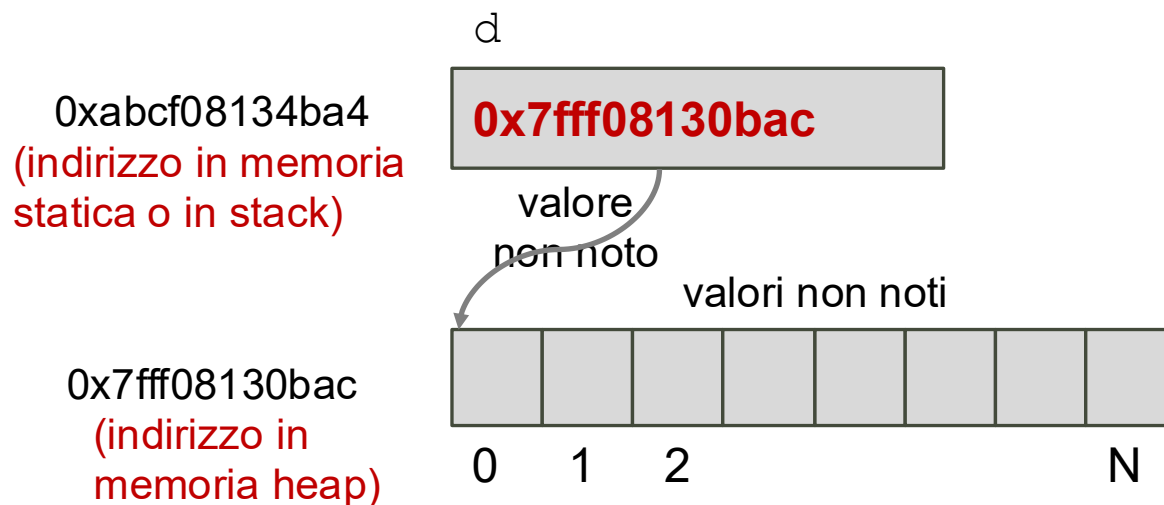
- Sì, esempio: `printf("%d", *(v+1))` // `*(v+1)` equivale a `v[1]`

- Posso scrivere `v++` ?

- No, `v` non può essere modificato

Allocazione Dinamica di un Array I

```
int *d;  
int N = 10;  
d = malloc(sizeof(*d) * N)
```



- Possiamo generalizzare?
 - Sì, quanto detto su array di `int` generalizza ad array di un qualsiasi tipo `T`

Allocazione Dinamica di un Array II

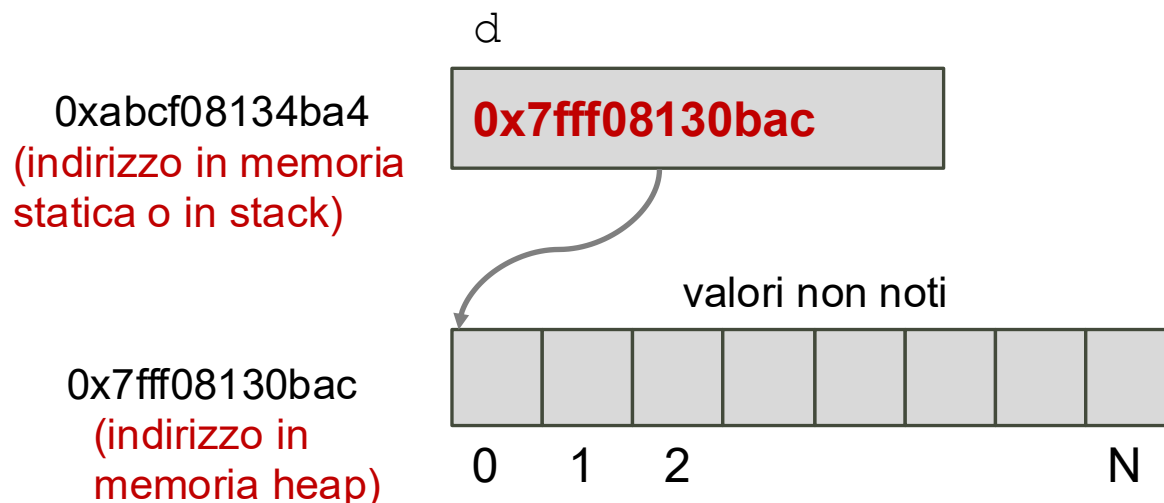


```
T *d;
```

```
int N = 10;
```

```
d = malloc(sizeof(*d) * N)
```

- Quanto spazio occupa?
 - Spazio per l'array ($N * \text{sizeof}(T)$) più puntatore (cioè + 8byte)
- Posso scrivere $v+1$?
 - Sì
- Posso scrivere $v++$?
 - Sì, vedi slide successiva



Allocazione Dinamica di un Array III

```
int *d;  
int N = 10;  
d = malloc(sizeof(*d) * N)
```

- Perché è scritto così?

```
d = (int *) malloc(sizeof(int) * N)
```

Uguale?

No, problema:

- Cosa succede se cambio `int *d` → `double *d`?

```
d = malloc(N * sizeof(*d))
```

Soluzione:

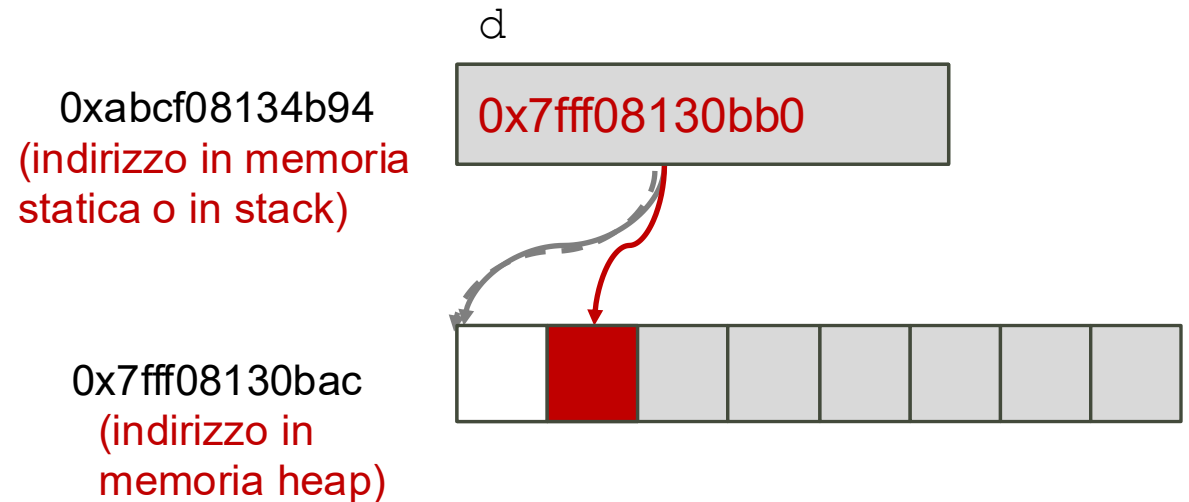
- Non dipende dal tipo di `*d`
- Rimane valido anche se `int *d` → `double *d`

Effetto di d++

```
T *d;
```

```
d = malloc(sizeof(*d) * N)
```

```
d++;
```



- Il valore di `d` è incrementato del numero di byte occupati da un elemento dell'array e punterà al secondo elemento
 - Esempio: Nel caso di elementi interi viene incrementata di 4 byte
 - L'elemento precedentemente in posizione 0 è accessibile attraverso l'espressione `d-1`

Deallocazione di un Array

```
T *d;
```

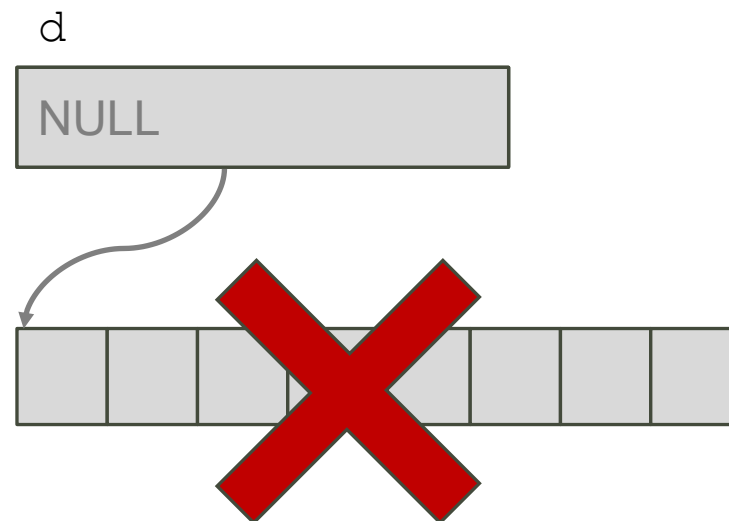
```
d = malloc(sizeof(T) * N)
```

```
free(d);
```

```
d = NULL;
```

0xabcf08134b94
(indirizzo in memoria
statica o in stack)

0x7fff08130bac
(indirizzo in
memoria heap)



- La memoria occupata dall'array sarà considerata (dal run-time del C) come non allocata e quindi (ri)allocabile
-  ATTENZIONE: il valore di `d` non diventa `NULL` per effetto della `free()`

Esempio di Uso Errato

```
char *d = (char *) malloc(sizeof(char)*10);
```

...

```
free(d); // senza mettere a NULL il puntatore
```

...

```
if (d != NULL) {  
    printf(..., d[0]);
```

...



-  Il valore **NULL** segnala che si riferisce ad un'area di **memoria non allocata**:
- Occorre **mantenere la consistenza**, altrimenti il valore di `d` non sarà significativo e l'esecuzione successiva sarà errata.



Passaggio di Array Come Parametro

```
int main() {  
    int astatico[N];  
    int *adinamico=(int *)malloc(sizeof(int)*N);  
  
    [init(astatico);  
     stampa(astatico);  
  
    [init(adinamico);  
     stampa(adinamico);  
}
```



L'effetto tra  e  sarà uguale o diverso?:

- Uguale perché anche il nome di un array statico corrisponde a un puntatore.

Allocazione di struct

Allocazione Statica/Automatica `struct`



```
typedef struct {  
    char titolo[MAXT];  
    int pagine;  
    char autore[MAXN];  
    float prezzo;  
} Libro;
```

Prototipo di struttura

- Non occupa memoria

```
Libro l;
```

Solo una dichiarazione del tipo di
struttura alloca memoria

Allocazione Dinamica struct

```
Libro *l;
```

```
l = malloc(sizeof(*l));
```

- Accesso ai campi?

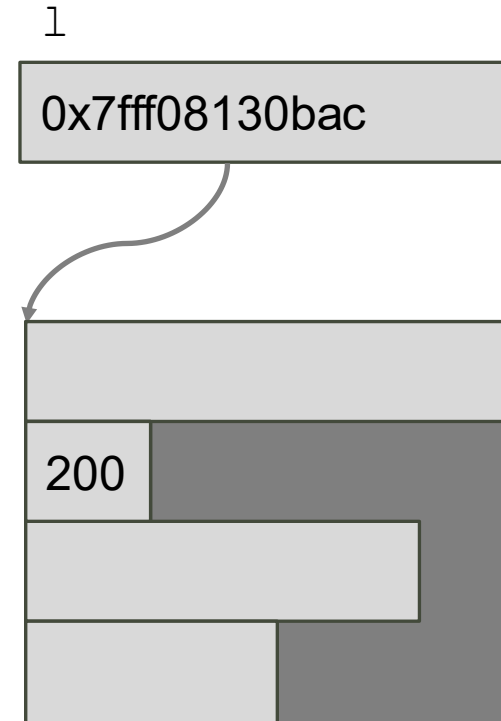
```
(*l).pagine = 200;
```

- Oppure più comunemente:

```
l->pagine = 200;
```

0x8abff09a40e00
(indirizzo in memoria
statica o in stack)

0x7fff08130bac
(indirizzo in
memoria heap)



Deallocazione Dinamica di struct

```
Libro *l;
```

```
l = malloc(sizeof(*l));
```

```
...
```

```
free(l);
```

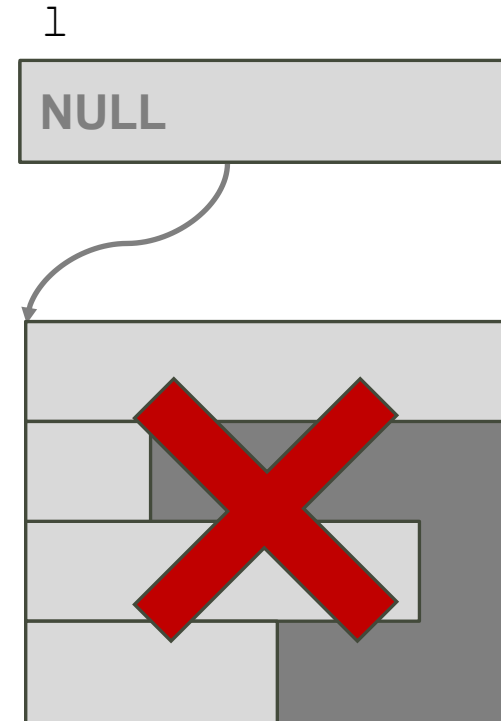
```
l = NULL;
```

Deallocazione:

- Necessarie entrambe per assicurare consistenza

0x8abff09a40e00
(indirizzo in memoria
statica o in stack)

0x7fff08130bac
(indirizzo in
memoria heap)



Passaggio di struct: Esempio



```
typedef struct {  
    int x;  
    int y;  
} Point;  
  
void stampa(Point p) { // non modifica p  
    printf("x=%d y=%d\n", p.x, p.y);  
}  
  
void init(Point *p) { // modifica p  
    p->x = 15;  
    p->y = 25;  
}
```

```
int main() {  
    Point automatica;  
    Point *dinamica = malloc(sizeof(*dinamica));  
    // inizializzo statica a 10, 10 e dinamica a 20, 20  
    automatica.x = 10; automatica.y = 20;  
    dinamica->x = 20; dinamica->y = 20;
```

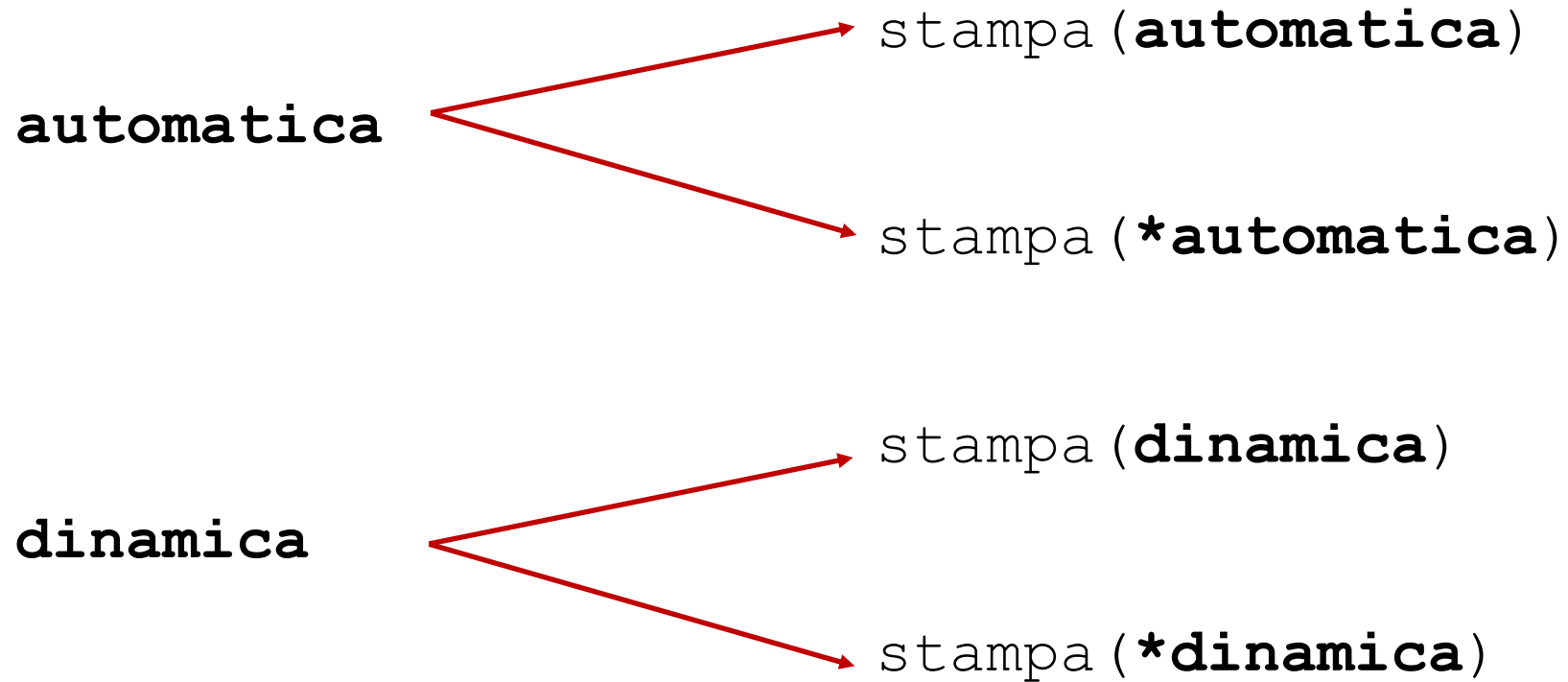
Come invoco sulle due variabili:

- stampa() ?
- init() ?

Passaggio di struct: Esempio



Come invoco `stampa()` sulle due variabili?

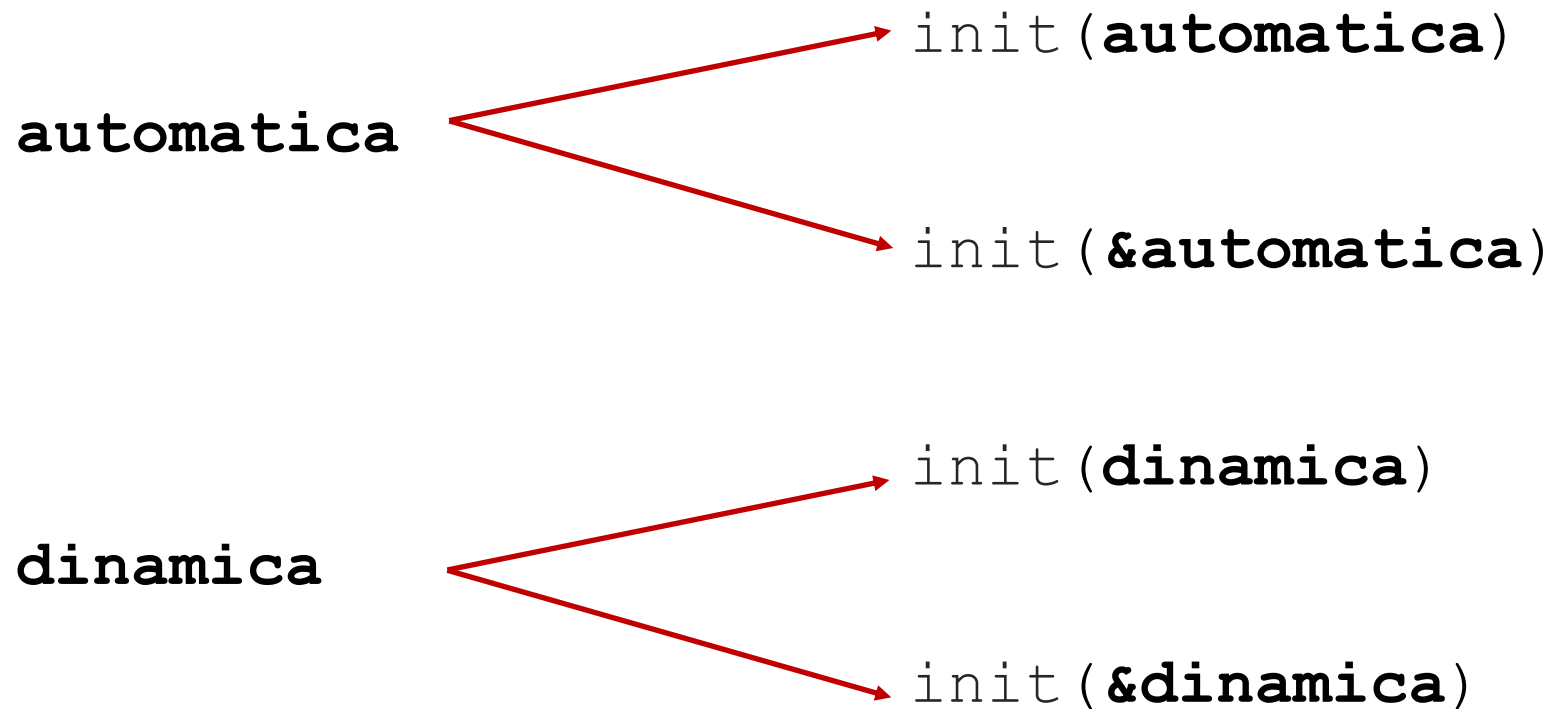


A differenza dagli array solo la seconda variabile è un puntatore

Passaggio di struct: Esempio



Come invoco `init()`
sulle due variabili?



Per modificare `automatica` occorre passare il suo indirizzo (puntatore a `automatica`):

- `init(&automatica)`

Invece `dinamica` è già l'indirizzo alla struct allocata dinamicamente:

- `init(dinamica)`

Quiz sui Tipi



- Quali delle seguenti istruzioni compilano senza errori di tipo?

- a. `p = l;`
- b. `p = &l;`
- c. `*p = &l;`
- d. `p->titolo = l->titolo;`

1. Qual è il tipo di `l.titolo[3]`?
2. E il tipo di `l.titolo`?
3. E il tipo di `&(l.pagine)`?

```
typedef struct {  
    char titolo[MAXT];  
    int pagine;  
    char autore[MAXN];  
    float prezzo;  
} Libro;  
  
Libro l;  
Libro *p = (Libro*)malloc(sizeof(Libro));
```

Quiz sui Tipi: Risposte I



- A una variabile è possibile assegnare il risultato di un'espressione che produce un valore del suo stesso tipo:
 - a. `p = l`
 - No, `p` è un puntatore a libro, `l` è un libro
 - b. `p = &l`
 - `&l` indica l'indirizzo di `l`, abbiamo due puntatori a libro quindi sì
 - c. `*p = &l`
 - `*p` deferenzia `p` quindi è un libro, `&l` è l'indirizzo di un libro (puntatore a libro), no
 - d. `p->titolo = l->titolo`
 - No, `l` non è un puntatore. E se fosse `p->titolo = l.titolo`?
 - Ancora no: i tipi corrispondono, ma gli array si assegnano così?

Quiz sui Tipi: Risposte II



1. Qual è il tipo di `l.titolo[3]`?

- `char`

2. E il tipo di `l.titolo`?

- `char*` (array di `char`)

3. E il tipo di `&(l.pagine)`?

- `int *`

Esercizio Consigliato



- Scrivere un programma che:
 1. Tramite `typedef` dichiara il nuovo tipo `Canzone`:
 - Una `struct` adatta a contenere informazioni su un brano musicale (es. titolo, autore, durata)
 2. Definisce tramite `#define` una costante `MAX`
 3. Alloca dinamicamente un array di `MAX` elementi di tipo `Canzone`
 - Non esagerate con `MAX`, bastano 3 elementi
 4. Esegue un ciclo in cui chiede all'utente le info sulle canzoni
 - E man mano li inserisce nell'array
 5. Visualizza a terminale i contenuti dell'array
 6. Dealloca l'array e termina

Funzioni Variadiche

Il prototipo di `printf`



- Il prototipo di `printf` è:

```
int printf(const char *format, ...);
```

- I puntini di sospensione (...) nel prototipo della funzione indicano un numero variabile di argomenti di qualsiasi tipo.
 - `printf` è una funzione variadica
- Funzioni Variadiche
 - Sono funzioni che accettano un numero variabile di argomenti di qualsiasi tipo

Funzioni Variadiche I



Come scrivo una funzione variadica?

- Prototipo:
 - Si usa la sintassi $T_0 \ f(T_1 \ x_1, \dots, T_n \ x_n, \dots);$
 - Dove T_n è il tipo del parametro x_n
 - La funzione deve avere almeno un parametro ($n \geq 1$) diverso dai puntini di sospensione
 - I puntini di sospensione devono essere l'ultimo parametro.
- Implementazione:
 - Basata sull'uso di `va_list`, `va_start`, `va_arg`, e `va_end` definite in `<stdarg.h>`
 - Gestiscono l'accesso ai parametri variabili salvati nello stack nel record di attivazione della funzione

Funzioni Variadiche II



- Implementazione:

1. Dichiaro una variabile di tipo di tipo `va_list`, e.g., `va_list ap;`
 - `ap` conterrà le informazioni necessarie per accedere all'elenco di argomenti a lunghezza variabile usando `va_start`, `va_arg`, e `va_end`
2. Inizializzo `ap` usando all'inizio della lista dei parametri variabili usando `va_start` e l'ultimo argomento fisso, e.g. `va_start(ap, Xn);`
3. Posso accedere in ordine a ciascun elemento della lista di $m \geq 0$ argomenti variabili, invocando ripetutamente `va_arg` specificando il tipo per ciascuno di essi, e.g., con lista di argomenti `double`:

```
for(int i =0; i< m; i++) {  
    double d = va_arg(ap, double); // va_arg() a ogni chiamata restituisce  
                                    l'argomento successivo  
}
```

4. Finito di accedere all'elenco di argomenti a lunghezza variabile, è necessario invocare `va_end`, e.g., `va_end(ap);`
 - Se `va_end` non viene chiamata prima di ritornare dalla funzione, il risultato non sarà definito.

Funzioni Variadiche: Esempio



```
#include <stdio.h>

double average(int n, ...); // calcolo
media di n double

double average(int n, ...) {
    double total = 0;
    va_list ap;
    va_start(ap, n); // init va_list

    // processa gli argomenti variabili
    for (int i = 0; i < n ; i++) {
        total += va_arg(ap, double);
    }
    va_end(ap); // cleanup va_list
    return total / n; // calcolo media
}
```

```
int main (void) {
    double w = 37.5;
    double x = 22.5;
    double y = 1.7;
    double z = 10.2;
    printf("%s%.1f %s%.1f %s%.1f %s%.1f\n",
        'W=', w, "x=", x, "y=", y, "z=", z);

    printf(" s%.3f\n %s%.3f\n",
        "Media di w e x è", average(2, w, x),
        "Media di w, x e y è", average(3, w, x, y);
}
```

Come mai funziona anche senza includere `<stdarg.h>`?

Come fa `printf` a sapere quale tipo usare con `va_arg()`?